RAG 실습 교안 · 2

Ollama 설치와 임베딩 모델 준비

문서의 글을 숫자 목록으로 바꾸는 실습 환경을 이 PC에 만듭니다

설치 대상	확인 항목	전체
Ollama 앱 · qwen3-embedding:0.6b	앱 준비 · 모델 준비 · Python 연결 · GPU 확인	10단계

RAG 실습 환경 준비: Ollama와 임베딩 모델

학생은 이 PC에서 문서의 글을 숫자 목록으로 바꾸는 환경을 준비합니다. **Ollama**는 임베딩 모델을 실행하고, **Python 프로그램**은 글을 보내 결과를 받습니다. 학생은 앱, 모델 파일, Python 연결, GPU 사용 근거를 차례로 확인합니다.

설치할 것 두 가지

① Ollama 앱 — 모델을 실행하고 요청을 받는 프로그램입니다. 공식 설치 파일로 설치합니다.

② 임베딩 모델 파일 `qwen3-embedding:0.6b` — 글을 비교할 숫자 목록으로 바꾸는 모델입니다. 명령으로 내려받습니다.

준비 뒤 확인할 일 네 가지(각각 다른 화면)

1. **앱 준비** · `ollama --version`
2. **모델 준비** · `ollama list`
3. **Python 연결** · `NB02` 코드 `1~5`
4. **GPU 확인** · `ollama ps` · 로그

네 가지 확인은 각각 다른 화면에서 합니다

설치 순서는 위에서 아래로 진행합니다. 각 줄의 '이 화면만으로 알 수 없는 것'이 다음 줄의 확인 이유입니다.

확인	어디서	학생이 하는 일	화면에서 읽을 것	이 화면만으로 알 수 없는 것
1. 앱 준비	PowerShell	`ollama --version` 입력	버전 숫자(예: 0.34.2)	모델·Python·GPU 준비
2. 모델 준비	PowerShell	`ollama pull qwen3-embedding:0.6b`, `ollama list` 입력	NAME 열의 qwen3-embedding:0.6b	Python 연결, GPU 사용
3. Python 연결	JupyterLab	NB02 코드 1~5 실행	`len(sample_vector)` → 1024	GPU 사용
4. GPU 확인	PowerShell	`ollama ps` 입력, 로그 확인	PROCESSOR 100% GPU, 장치 이름	다른 PC의 상태

NB02는 실습 노트북 `02_Ollama_임베딩과_ChromaDB_저장.ipynb`의 약칭입니다. '코드 1~5'는 설명 셀을 뺀 코드 셀 순서입니다.

> **기준** 한 단계가 끝났어도 다음 단계가 자동으로 성공한 것은 아닙니다. 학생은 네 화면을 각각 확인합니다.

다음 → 임베딩 모델이 무엇을 받아 무엇을 만드는지 먼저 봅니다

글을 숫자로 바꾸는 qwen3-embedding:0.6b

임베딩 모델은 글을 비교할 숫자 목록으로 바꾸는 모델입니다. 답변 문장을 쓰거나 문서를 요약하는 모델이 아닙니다.

설명용 도식 — 실제 입력과 실제 결과 개수를 표시했습니다

실제 입력(첫 글 조각)

ID 2025FY_0001

“SK하이닉스 2025FY / 공시 개요 / # 사업보고서”

→

Python 프로그램

코드 4에서 `ollama.embed`로

이 글을 Ollama에 보냅니다

→

Ollama 앱

요청을 받아 임베딩 모델을

실행하고 결과를 돌려줍니다

→

임베딩 모델

`qwen3-embedding:0.6b`

글을 숫자 목록으로 바꿉니다

→

결과: 숫자 목록

이 입력의 결과는 숫자 1,024개

예: 0.080, 0.0017, -0.0070, …

이번 실습은 한국어 자료도 다루는 **qwen3-embedding:0.6b**를 사용합니다. 이름 끝의 **0.6b**는 모델의 크기(매개변수 수) 표시이고, 파일 크기 639 MB와는 다른 숫자입니다.

‘예: 0.080, 0.0017, -0.0070, …’는 노트북 설명 셀에 저장된 값의 일부입니다. 숫자 하나하나를 읽는 것이 아니라 목록 전체를 다른 글의 목록과 비교하는 데 씁니다. **Ollama**(모델을 실행하는 프로그램)와 **임베딩 모델**(Ollama가 실행하는 파일)은 다른 대상입니다.

다음 실습

다음 교안에서 Python 프로그램은 이 숫자를 ChromaDB에 저장합니다. 질문도 같은 모델로 숫자로 바꿔야 ChromaDB가 같은 기준으로 비교할 수 있습니다.

다음 → 이 교안에서 각 담당자를 부르는 이름을 정리합니다

이 교안에서 부르는 이름

같은 것을 여러 이름으로 부르지 않도록 정했습니다. 문장에서 담당자가 바뀌면 이름을 다시 씁니다.

이름	이 교안에서 맡는 일
학생	창 열기, 명령 입력, 셀 실행, 결과와 GPU 상태 확인을 합니다
PowerShell	학생이 입력한 명령을 실행하는 터미널 창입니다
Ollama 앱·명령	모델 다운로드·실행 요청을 처리하고 상태를 보여 주는 프로그램입니다
Python 프로그램	실습 파일을 읽고 Ollama에 임베딩을 요청하는 노트북 코드입니다
Python의 ollama 패키지	Python 코드가 Ollama에 요청하도록 돕는 도구입니다(`import ollama`)
임베딩 모델	글을 비교할 숫자 목록으로 바꾸는 `qwen3-embedding:0.6b`입니다
GPU 장치	모델의 계산을 처리하는 하드웨어입니다(이 PC: NVIDIA GeForce GTX 1080 Ti)
ChromaDB	다음 교안에서 글·숫자·출처를 저장하고 검색하는 도구입니다

> **구분** ‘Ollama 앱’과 ‘Python의 ollama 패키지’는 다른 설치물입니다. 버전도 따로입니다(이 PC: 앱 0.34.2, 패키지 0.6.2).

다음 → 사용할 PC의 Windows와 GPU 장치를 확인합니다

Windows용 Ollama 설치 파일 내려받기

학생은 브라우저에서 Ollama 공식 다운로드 페이지를 엽니다. 이 파일은 Ollama 앱을 설치하는 파일입니다. 임베딩 모델 파일은 뒤에서 따로 받습니다.

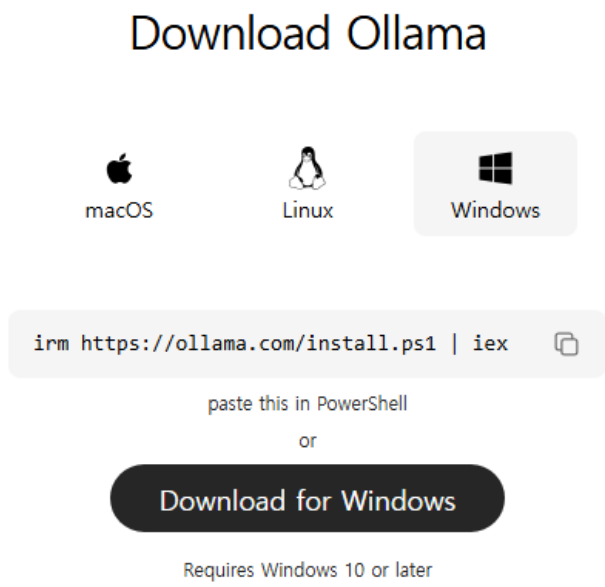
```
https://ollama.com/download/windows
```

① 페이지 가운데의 **Windows**를 선택합니다(캡처에서 회색으로 표시된 칸).

② **Download for Windows** 단추를 누릅니다. 브라우저가 설치 파일 **OllamaSetup.exe**를 PC에 저장합니다.

③ 단추 아래 'Requires Windows 10 or later'는 Windows 10 이상이 필요하다는 뜻입니다.

단추 위의 한 줄 명령(`irm https://ollama.com/install.ps1 | iex`)은 같은 앱을 PowerShell로 설치하는 다른 방법입니다. 이 교안은 단추로 받는 방법을 기준으로 합니다.



> **구분** 이 페이지는 설치 완료 화면이 아닙니다. 파일을 받은 뒤 다음 쪽에서 설치합니다.

다음 → 받은 파일로 앱을 설치하고 실행 상태를 확인합니다

Ollama 앱 설치와 실행

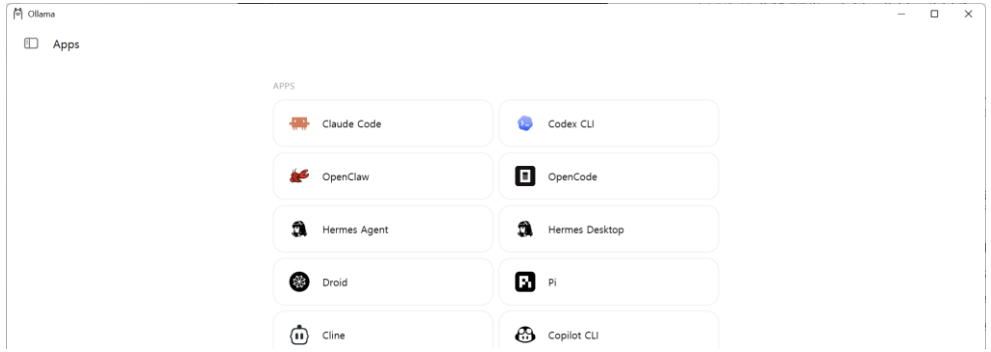
학생은 내려받은 OllamaSetup.exe를 열고 화면의 안내에 따라 설치합니다. 일반 Windows 설치는 사용자 계정에 앱을 설치합니다.

설치: 관리자 권한은 필요하지 않습니다. 설치 파일이 묻는 대로 진행하면 Ollama 앱이 사용자 계정에 설치됩니다.

실행: 설치가 끝나면 Ollama는 화면 뒤(백그라운드)에서 요청을 받을 수 있습니다. 작업 표시줄 오른쪽 알림 영역에 Ollama 아이콘이 보입니다.

앱 창: 시작 메뉴에서 Ollama를 열면 오른쪽 같은 창이 뜹니다. 이 창 안의 목록은 이번 실습에서 쓰지 않습니다. 창을 닫아도 앱은 뒤에서 계속 실행됩니다.

이 교안의 화면: 이미 설치된 교육 PC에서 앱 창을 연 모습입니다. 설치 진행 창은 이 PC에서 다시 촬영하지 못했습니다. 학생의 PC에서는 설치 창의 단추 이름이 버전에 따라 조금 다를 수 있습니다.



주의 앱 창이 보인다는 것과 모델이 준비됐다는 것은 다릅니다. 다음 쪽에서 PowerShell 명령이 실행되는지 확인합니다.

다음 → 새 PowerShell 창에서 ollama 명령이 되는지 확인합니다

PowerShell에서 Ollama 명령 확인

PowerShell은 명령을 입력해 프로그램을 실행하는 창입니다. 학생은 PowerShell을 새로 열고 아래 명령을 입력합니다.

```
ollama --version
```

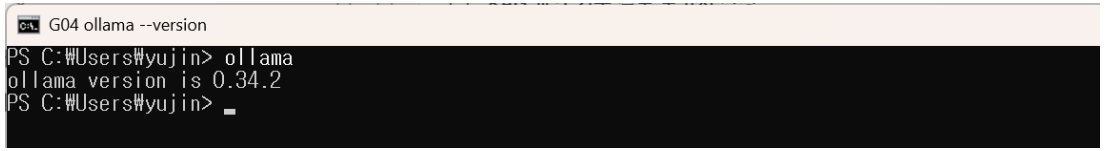
장소: 시작 메뉴에서 PowerShell을 새로 엽니다. 설치 전에 열려 있던 창은 새 명령을 모를 수 있습니다.

행동: `ollama --version`을 입력하고 Enter를 누릅니다.

처리: Ollama가 자기 버전을 확인합니다.

화면: `ollama version is 0.34.2`처럼 버전 숫자가 나옵니다. 학생의 PC는 숫자가 다를 수 있습니다.

판정: 숫자가 나오면 Ollama 명령이 실행된 것입니다. ‘ollama 용어를 인식하지 못한다’는 메시지가 나오면 새 창에서 다시 입력하고, 그래도 안 되면 설치를 확인합니다.



화면 읽기

첫 줄은 학생이 입력한 명령입니다. 둘째 줄 `ollama version is 0.34.2`가 Ollama의 대답입니다.

셋째 줄의 `PS C:\Users\…>`는 다음 명령을 기다리는 표시입니다. 오류가 있으면 이 자리 앞에 빨간 글씨가 나옵니다.

구분 0.34.2는 Ollama 앱 버전입니다. Python 패키지 버전(0.6.2)과는 별개입니다. 버전만으로 모델·Python·GPU 준비를 확인한 것은 아닙니다.

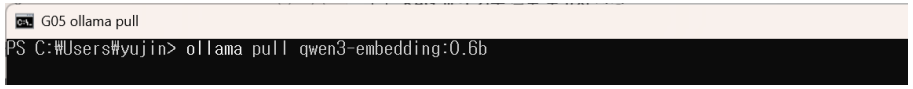
다음 → 다음 확인: 모델 파일 준비

정확한 이름으로 임베딩 모델 내려받기

학생은 PowerShell에 아래 명령을 입력합니다. pull은 모델 파일을 받으라는 명령입니다. Ollama는 이 이름에 맞는 모델 파일을 준비합니다.

```
ollama pull qwen3-embedding:0.6b
```

명령의 부분	뜻
ollama	Ollama에게 시키는 명령이라는 표시입니다
pull	모델 파일을 이 PC로 받으라는 뜻입니다
qwen3-embedding	모델 이름입니다
:0.6b	콜론 뒤는 태그(크기 표시)입니다. 이번 실습은 반드시 0.6b를 씁니다



입력 확인

Enter를 누르기 전에 `qwen3-embedding:0.6b`의 철자, 하이픈(-), 콜론(:)을 확인합니다.

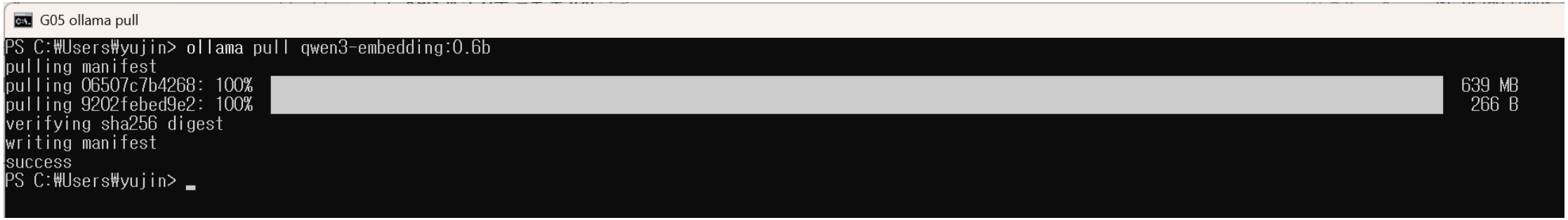
정확한 이름은 다음 교안의 노트북 코드에도 같은 글자로 들어 있습니다(코드 3의 `model`).

주의 이 명령은 PowerShell에서 입력합니다. 노트북의 Python 셀에 넣는 명령이 아닙니다. 이름을 줄이거나 4b·8b로 바꾸면 다른 모델을 받게 됩니다.

다음 → Enter를 누른 뒤 진행 화면을 읽습니다

모델 다운로드 진행 화면 읽기

Ollama는 모델 파일을 받는 동안 진행 상태를 보여 줍니다. 학생은 마지막 줄에 success가 나올 때까지 기다립니다.



화면 읽기

`pulling …: 100%` 두 줄은 모델을 이루는 파일 두 개(639 MB, 266 B)의 진행 막대입니다. 두 줄 모두 100%가 되어야 합니다.

`verifying sha256 digest`는 받은 파일이 깨지지 않았는지 확인, `writing manifest`는 모델 정보 기록입니다.

이 화면과 학생 PC의 차이

이 화면은 파일이 이미 있는 교육 PC라서 인터넷 전송 없이 바로 100%로 끝났습니다.

처음 받는 PC에서는 막대가 천천히 채워지고 몇 분이 걸릴 수 있습니다. 끝나는 시간은 PC마다 다릅니다.

판정 마지막 줄 success가 다운로드 완료입니다. 이 화면은 모델 파일 준비를 보여 줄 뿐, GPU 사용을 보여 주지 않습니다.

다음 → 완료 뒤 모델 목록에서 이름을 확인합니다

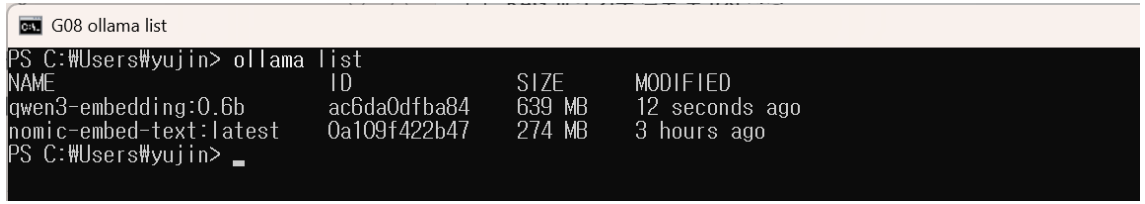
다운로드 완료와 모델 목록

학생은 ollama list를 입력합니다. Ollama는 이 PC에 준비된 모델 목록을 보여 줍니다. 학생은 NAME 열에서 이번 실습의 모델을 찾습니다.

```
ollama list
```

열	뜻	이 PC의 값
NAME	모델 이름과 태그	qwen3-embedding:0.6b
ID	모델 파일의 식별값	ac6da0dfba84
SIZE	모델 파일의 크기	639 MB
MODIFIED	파일이 준비된 시점	12 seconds ago

MODIFIED는 PC마다 다르게 보입니다. ID는 같은 모델이면 같은 값입니다.



판정

NAME 열에 `qwen3-embedding:0.6b`가 있으면 필요한 모델 파일이 준비된 것입니다.

다른 모델이 함께 보여도 괜찮습니다. 태그가 다른 모델(예: 4b)은 같은 모델이 아닙니다.

GPU 사용은 이 목록으로 알 수 없습니다(I12에서 확인).

구분 list는 '파일이 있는가', ps는 '지금 메모리에 올라가 있는가'를 보여 줍니다. SIZE는 파일 크기이지 GPU 메모리 사용량이 아닙니다.

다음 → 다음 확인: Python 프로그램이 이 모델에 요청할 수 있는지